



ICS3U Resource Book

Revised August 2013

10.1 Sequential Search

To begin, suppose that you are looking for a particular CD in your collection. After rummaging around for a while with no luck, you may decide to go through the collection systematically, looking at each CD in turn. If you are patient, you will eventually find the one that you are looking for (if it is there).

This simple technique, called a *sequential search*, can be applied to an array of values.

Example 1

The following method performs a sequential search on an array of `String` values for the value called `item`. If the search is successful, the method returns the index in the array of `item` but, if the search fails, the method returns `-1`.

```
public static int seqSearch (String[] list, String item)
{
    int location = -1;
    for (int i = 0; i < list.length; i++)
        if (list[i].equals(item))
            location = i;
    return location;
}
```

Note that the variable `location` is initialized to indicate that the search is unsuccessful. If `item` is not found, this is the value that the method will return. Only if we find `item` in the list will `location` be changed.

Although this method works, it is not as efficient as we might wish. Even if it finds `item` early in the search, it stupidly goes on looking through the rest of the array. We can correct this defect by using a `boolean` variable that acts as a flag to stop the search as soon as `item` has been found. We use this device in the next example.

Example 2

This method shows an efficient implementation of sequential search that quits examining a list immediately after the search has found `item`.

```
public static int seqSearch (String[] list, String item)
{
    int location = -1;
    boolean found = false;
    for (int i = 0; i < list.length && !found; i++)
        if (list[i].equals(item))
        {
            location = i;
            found = true;
        }
    return location;
}
```

We can achieve even greater efficiency for our sequential search, as shown in the next example.

Example 3

By eliminating the boolean flag `found` and the corresponding test in the `for` statement to determine the state of `found`, we can create the following very efficient sequential search.

```
public static int seqSearch (String[] list, String item)
{
    for (int i = 0; i < list.length; i++)
        if (list[i].equals(item))
            return i;
    return -1;
}
```

Now, if a match for `item` is ever found, we exit both the `for` statement and the method immediately. Only if no match is ever found do we complete the `for` statement and return `-1`.

Although the code of Example 3 is both simpler and more efficient than that of Example 2, some people still argue that the solution in Example 2 is preferable. In Example 3, the first line of the `for` statement would make a reader think that the loop is going to be executed for every value in the array but, if a match for `item` is found, this does not happen. In Example 2, on the other hand, the first line of the `for` statement is quite clear about what is going to happen — the loop will be executed as long as we haven't reached the end of the array and we haven't found what we are seeking.

Exercises 10.1

1. Suppose that the method of Example 1 was used to search an array. What would the method return if `item` appeared in the list more than once?
2. What changes should be made to the sequential search in Example 2 so that it searches an array of values starting at the top and moving downward?
3. A modification of the basic sequential search operates in the following way. If the item being sought is found, it is interchanged with the item that preceded it. If, for example, we were searching for 7 in the list

3 9 5 7 2 8 4

then, after finding 7, the list would be rearranged in the order

3 9 7 5 2 8 4

- (a) Write a method that implements this technique to search an array of `int` values.
- (b) Test your method in a program that first asks for the length of the list to be searched and then reads that many integers into an array. The program should then repeatedly prompt the user for values until the user supplies a sentinel of zero. The program should print the initial list and then, for each non-zero value read, it should use your modified sequential search to try to locate that item and then print the resulting array.
- (c) Why might this modification sometimes improve the efficiency of a sequential search?

10.2 Binary Search

Suppose that you are, once again, looking for a CD in your collection. If you have gone to the trouble of putting the CD's in order, your search can be speeded up considerably and you should be able to locate the one that you want very quickly. Our next algorithm, called a *binary search*, provides an efficient method for searching a list in which the items are ordered. The algorithm is an example of a large class called *divide and conquer* algorithms. These algorithms employ strategies that solve the problem by quickly reducing its size. For a binary search, at each stage of the solution, we cut the size of the problem roughly in half.

To illustrate the way that the algorithm works, suppose that we are searching for the item 47 in the sorted list shown below.

16 19 22 24 27 29 37 40 43 44 47 52 56 60 64 71

To start the process, we initially examine the item in the middle of the array. The middle item, 40, is not the one that we want but, because it is less than the value that we are looking for and because the list is sorted, we know that we can eliminate *all* the items in the lower half of the list. Our search can now continue looking only at the remaining half of the original list, as shown.

43 44 47 52 56 60 64 71

We now repeat our strategy on these items. Since there are now eight items left, there is no exact middle; we can choose either 52 or 56. If we look at 52, we see that it is larger than the value for which we are searching. Again taking advantage of the fact that the items are ordered, we can eliminate 52 and all items above it from our search. This leaves us with only the following values.

43 44 47

We again look at the middle item, this time finding 44, which is too small. We can therefore discard both it and the item below it, leaving us with only one value.

47

One final probe, looking at 47, produces a successful search. In only four comparisons, we have been able to find the item that we want.

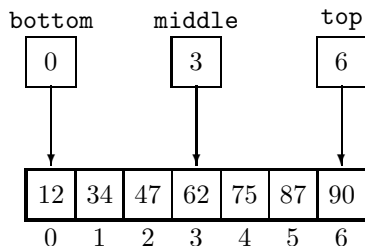
Let us now see how we can implement a binary search in Java. Suppose that we are searching for `item` in an array called `list`. At the start of the search, `item` could be anywhere in `list` but, as the search proceeds, the interval that is being searched is reduced over and over again. To keep track of the bounds of this interval, we use two `int` variables: `bottom`, containing the index of the lower bound of the current interval and `top`, containing the index of the upper bound. The easiest way to find the index of the middle (or near middle) of the interval is to take the average of `bottom` and `top`. Once we have found the middle, we examine the value located there. If it is equal to `item`, the value we are seeking, we are done. If it is smaller than `item`, we know that we can discard both the value at `middle` and all values below this. To do so, we simply change the value of `bottom` to `middle + 1`. Similarly, if the value located at `middle` is larger than `item` we discard the upper values of the interval by setting `top` to `middle - 1`.

Example 1

Suppose that we want to perform a binary search for the value 75 on the following data.

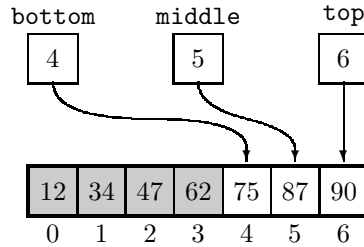
12 34 47 62 75 87 90

Initially, we want to examine the entire array so the variables `bottom` and `top` are initialized to 0 and 6, the indices of the first and last elements, respectively while `middle` is set to $(0 + 6)/2 = 3$. (In the diagrams, we use arrows to emphasize the purpose of `bottom`, `middle`, and `top`; they should not be confused with the arrows that we use for reference variables.)

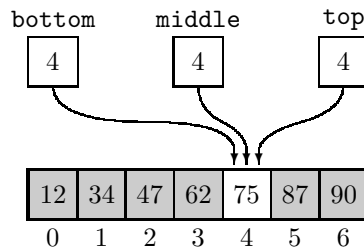


Since $62 < 75$, the item that we are seeking cannot be in the left half of the list. We discard this half of the list by setting `bottom` to `middle + 1`.

The middle of the remaining interval is $(4 + 6)/2 = 5$, as shown in the next diagram.



Since $87 > 75$, the value 75 cannot be in the upper half of this sublist so we discard it by setting `top` to `middle - 1`. Since `bottom` and `top` are now both equal to 4, the value of `middle` will be $(4 + 4)/2 = 4$.



Once `middle` has found the value, the search ends successfully.

The next example implements a binary search for the value `item` in an array `list` of `double` values. It returns the index in `list` of `item` (if `item` is in `list`) and -1 otherwise. Notice how the search terminates if the item that we are looking for is *not* in the list. We saw in the last example how the values of `bottom` and `top` converge as the search proceeds with `bottom` getting larger and `top` getting smaller. If the item that we are seeking is not in the list, eventually we will either set `bottom` to a value larger than `top` or we will set `top` to a value smaller than `bottom`. Thus, we only keep searching as long as $\text{bottom} \leq \text{top}$ and the item that we are seeking has not been found.

Example 2

```
public static int binSearch (double[] list, double item)
{
    int bottom = 0;           // lower bound of subarray
    int top = list.length - 1; // upper bound of subarray
    int middle;               // middle of subarray
    boolean found = false;    // to stop if item found
    int location = -1;        // index of item in array

    while (bottom <= top && !found)
    {
        middle = (bottom + top)/2;
        if (list[middle] == item) // success
        {
            found = true;
            location = middle;
        }
        else if (list[middle] < item) // not in bottom half
            bottom = middle + 1;
        else // item cannot be in top half
            top = middle - 1;
    }
    return location;
}
```

Exercises 10.2

1. Suppose that an array contains the following elements.

23 27 30 34 41 49 51 55 57 60 67 72 78 83 96

Trace the execution of the method `binSearch` shown in this section as it searches for the following values of `item`. In each trace, show

the progress of the search by using diagrams similar to those in Example 1.

(a) 72 (b) 41 (c) 62

2. What changes would have to be made to `binSearch` so that it will search an array that is sorted in *descending* order.
3. Rewrite `binSearch` so that, if a search is unsuccessful, the method will return the index of the value *nearest* to `item`, instead of returning `-1`. (If there is a tie, return the smaller index.)
4. What is the maximum number of comparisons that might be necessary to perform a binary search on a list containing seven items?
5. Repeat the previous question for lists with the indicated sizes.

(a) 3	(b) 15	(c) 31	(d) 63
(e) 100	(f) 500	(g) 1 000	(h) 10 000

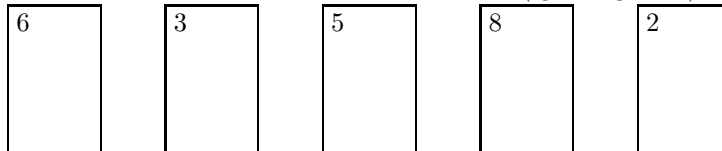
10.3 Insertion Sort

To use a binary search, we noted that it was necessary to have the items sorted. We now turn our attention to finding ways of sorting data.

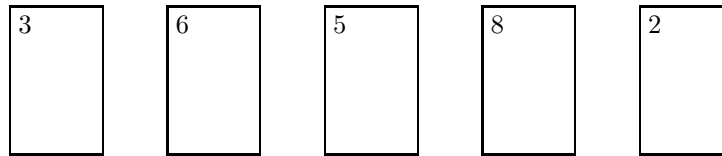
To develop algorithms of any kind, it is often useful to examine the ways that we solve similar problems in everyday life. To develop a solution to the problem of sorting data, let us look at how we might proceed if we were playing a card game and we wanted to put our hand in order, from smallest to largest. A common way of sorting such a hand is to examine cards from left to right, rearranging them if necessary by placing cards where they belong in the sequence of cards on their left. The next example illustrates the process.

Example 1

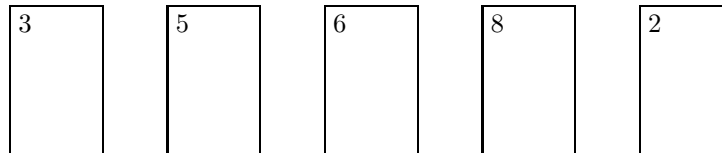
Suppose that we have the five cards shown below (ignoring suits).



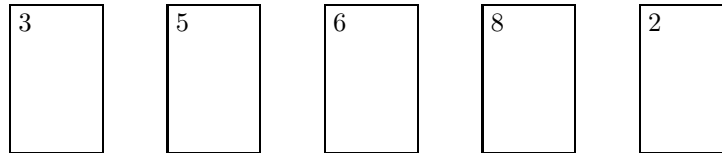
We could begin to sort these cards by looking at the 3, the second card from the left. Since $3 < 6$, the 3 should be inserted to the left of the 6. This will produce the following arrangement in which the two values on the left are in order.



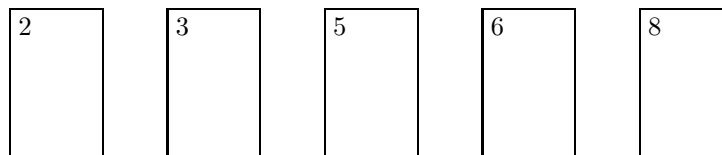
The next card from the left, the 5, should be inserted between the 3 and the 6. Doing this gives us the next arrangement in which the first three cards are guaranteed to be in order.



Now we examine the 8. Because it is greater than any of the values to its left, it should stay where it is and still give us the four leftmost values in order.



Finally, looking at the 2, we can see that it must be inserted before any of the other values. Doing this gives us our final, ordered arrangement.



To implement this algorithm for an array of values, we must examine and correctly insert each of the values in the array from the second position up to the last one. This requires a loop like the following, for an array called `list`.

```

for (int top = 1; top < list.length; top++)
    // insert element at top into its correct position
    // among the elements from 0 to top - 1

```

At each stage or *pass* of the sort, we first copy the element that we want to insert into a temporary location. We then move from right to left through the (already sorted) items on the left of the value that we are inserting. If a value in the list is larger than the new one, it is moved one space to the right (to provide room for the new item). Once the correct location of the new value is found, it is then inserted back into the array from the temporary location in which it had been saved. These ideas are incorporated into the method of the next example.

Example 2

The method `insertSort` uses an insertion sort to arrange an array of `double` values in ascending order.

```

public static void insertSort (double[] list)
{
    for (int top = 1; top < list.length; top++)
    {
        double item = list[top];
        int i = top;
        while (i > 0 && item < list[i-1])
        {
            list[i] = list[i-1];
            i--;
        }
        list[i] = item;
    }
}

```

Exercises 10.3

1. An insertion sort is to be used to put the values

6 2 8 3 1 7 4

in ascending order. Show the values as they would appear after each pass of the sort.

2. What changes would have to be made to the `insertSort` method in Example 2 in order to sort the values in descending order?
3. What might happen if, in Example 2, the `while` statement's first line were written in the following form?

```
while (item < list[i-1] && i > 0)
```
4. Write a program that initializes an array with the names of the planets ordered by their distances from the sun (Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune, and Pluto) and prints them in that order on one line. The program should then use an insertion sort to arrange the names alphabetically. To trace the progress of the sort, have it print the list after each pass.
5. The *median* of an ordered list of numerical values is defined in the following way. If the number of values is odd, the median is the middle value. If the number of values is even, the median is the average of the two middle values. Write a program that first prompts the user for the number of items to be processed, reads that many real values, and then finds their median.
6. A sort is said to be *stable* if it always leaves values that are considered to be equal in the same order after the sort. Is the insertion sort stable? Justify your answer.

10.4 Selection Sort

In using an insertion sort, values are constantly being moved. A sorting technique that reduces the amount of data movement is the *selection sort*. Here, we again perform a number of passes through the data and, after each pass, we again place one more item into an ordered sequence. Now, however, once an item has been placed in its position in the ordered sequence, it is never moved again.

We can begin a selection sort by scanning all the values, finding the largest one, and placing it at the top of the list, exchanging it with the item that was originally in this position.

Example 1

To begin sorting the data

7 1 9 3 5 4

we find the largest element, 9, and swap it with 4, the element at the right end of the list. The result, after the first pass of a selection sort is

7 1 4 3 5 9



On the second pass, all the items except the last are examined to see which of these is the largest and this item is then placed at the right end of this sublist. This pattern continues on subsequent passes; on each one, the largest value among the unsorted items is placed at the top of the sublist.

Example 2

Using the set of data shown in Example 1, the table shows the results of successive passes of selection sort. The vertical bars indicate the division points between the unsorted items and the sorted items.

After Pass

1	7	1	4	3	5		9
2	5	1	4	3		7	9
3	3	1	4		5	7	9
4	3	1		4	5	7	9
5		1	3	4	5	7	9

Notice that only five passes are required to sort six items. Once all but one of the items are placed in their correct positions, the remaining item must also be in its correct position.

To code this algorithm in Java, we note that, for an array called `list`, we must successively find the largest item in sublists of sizes `list.length`, `list.length-1`, ..., 2. Since the upper bound of an array has index that is one less than the size of the array, the loop that controls the passes of the sort will have the form

```
for (int top = list.length - 1; top > 0; top--)  
    // locate largest item and then  
    // swap it with item at list[top]
```

We have already seen how to find the largest element in an array and how to swap elements in an array. The next example shows the selection sort that results when we put these pieces together.

Example 3

The method `selectSort` uses a selection sort to arrange an array of double values in ascending order.

```
public static void selectSort (double[] list)  
{  
    for (int top = list.length - 1; top > 0; top--)  
    {  
        int largeLoc = 0;  
        for (int i = 1; i <= top; i++)  
            if (list[i] > list[largeLoc])  
                largeLoc = i;  
  
        double temp = list[top];  
        list[top] = list[largeLoc];  
        list[largeLoc] = temp;  
    }  
}
```

Exercises 10.4

1. If a selection sort were to be used to sort the data shown below in alphabetical order, show the data after each pass of the sort.

Renée Brien Vincent Doris Scarlett

2. In the `selectSort` method shown in Example 3, what would happen if the expression `list[i] > list[largeLoc]` were to be changed to `list[i] < list[largeLoc]`?

3. In our version of selection sort, if the largest item is already at location `top` in the list, then the method still swaps that value with itself, even though that is not necessary.
 - (a) How could the method be changed to avoid this unnecessary swapping?
 - (b) Why might it be better to leave the method as it is in the text?
4. On each pass of our version of selection sort, the *largest* value among the remaining unsorted items was placed in its correct position. An alternate form of the algorithm uses each pass to place the *smallest* value among the remaining unsorted values in its correct position.
 - (a) Given the set of data

8 9 6 1 2 4

show the data as they would appear after each pass of a selection sort using this algorithm.
 - (b) Write a Java method that implements this algorithm to sort an array of `int` values.
5. Sometimes we are only interested in knowing the values that would occupy one end of the list if the list were sorted. As an example, we may want to know the scores of only the top ten competitors in a contest. Modify the selection sort of Example 3 so that, instead of sorting the entire array, it puts the k largest values in order in the last k positions in the array. The value of k should be a parameter of the method.

10.5 Bubble Sort

Our next (but not our last) sorting algorithm, called a *bubble sort*, is an example of a class of sorts known as *exchange sorts*. With the bubble sort, the basic idea is to compare adjacent values and exchange them if they are not in order.

Example 1

Suppose that we want to use a bubble sort to arrange the names

Phil Ivan Sara Jack Gina

in alphabetical order. We start by comparing Phil to Ivan. Since they are not in order, we exchange them to give

Ivan Phil Sara Jack Gina

Next we compare Phil to Sara. Since they are in order, we leave them that way giving

Ivan Phil Sara Jack Gina

Now, comparing Sara to Jack, we see that they must be exchanged. Doing this gives

Ivan Phil Jack Sara Gina

We then compare Sara to Gina. Again, we must perform an exchange to obtain

Ivan Phil Jack Gina Sara

The result of all this comparing and exchanging is that, after one pass, the largest value (Sara, in our example) will be at the upper end of the list. Like a bubble rising in a liquid, the largest value has risen to the top of the list.

As with the selection sort shown in the previous section, we now repeat the process used in the first pass on all but the last element. As we proceed, the passes deal with shorter and shorter subsequences of the original list until all values are in their correct positions.

Example 2

The following tables show the actions of a bubble sort in ordering the names

Phil Ivan Sara Jack Gina

The colons indicate the values currently being compared. The vertical bars indicate the division points between the unsorted data and the sorted data.

	Phil	:	Ivan	Sara	Jack	Gina
	Ivan		Phil	:	Sara	Jack
First Pass	Ivan		Phil		Sara	:
	Ivan		Phil	Jack	Sara	:
	Ivan		Phil	Jack	Sara	:
	Ivan		Phil	Jack	Gina	
						Sara
	Ivan	:	Phil	Jack	Gina	
Second Pass	Ivan		Phil	:	Jack	Gina
	Ivan		Jack		Phil	:
	Ivan		Jack	Gina		Phil
						Sara
	Ivan	:	Jack	Gina		Phil
Third Pass	Ivan		Jack	:	Gina	
	Ivan		Gina		Jack	Phil
						Sara
Fourth Pass	Ivan	:	Gina		Jack	Phil
	Gina		Ivan		Jack	Phil
						Sara

As we saw with selection sort, the number of passes needed to sort the items is one less than the number of items. After all but one of the items have been ordered, the remaining one must also be in its correct position.

To create a Java method for a bubble sort is fairly easy if we use some of our previous techniques. To perform all the passes we need a loop like that of the selection sort.

```
for (int top = list.length-1; top > 0; top--)
    // compare adjacent values of the unsorted sublist
    // from 0 to top, exchanging as necessary
```

Within each pass, the code for comparing and exchanging values in a sublist is easily written.

```
for (int i = 0; i < top; i++)
    if (list[i].compareTo(list[i+1]) > 0)
    {
        temp = list[i];
```

```
list[i] = list[i+1];  
list[i+1] = temp;  
}
```

Combining these fragments of code and inserting them into a method gives us a basic bubble sort. We can improve this sort's performance by noting that, on each pass, we often do more than simply place the greatest value at the upper end of a sublist; the comparisons and exchanges tend to push *all* values toward their final positions in the list. It may happen that this shuffling of values puts all the values in their final positions before all passes have been completed. In such a situation, we should stop working as soon as possible, avoiding any unnecessary passes. As it turns out, this is not too difficult to do.

Before we incorporate this modification, we must first answer the following question: how do we know if there is no more work to be done? The answer is: if *no* exchanges are performed in an entire pass, then the items must be fully sorted. The reasoning here is that, if no exchanges are needed, then each value must be correctly ordered between its immediate neighbours. It follows that the entire set of values must, therefore, be completely ordered. Thus we need to perform another pass only if the previous one carried out one or more exchanges.

To encode this idea in Java, the `for` statement that controls the number of passes should now incorporate a test to see if another pass is necessary. We do this with a `boolean` variable called `sorted`. Initially, `sorted` is set to `false`; we do not start a pass unless `sorted` is `false`. Once we have started a pass, we (optimistically) set `sorted` to `true` because no exchanges have yet been performed during that pass. If any exchanges are required during a pass, we reset `sorted` to `false`. Processing continues until `sorted` remains `true` after a full pass or we have completed the maximum number of passes.

Example 3

This method uses a bubble sort to place an array of strings in ascending order. It uses a boolean flag, `sorted`, to make the bubble sort more efficient by stopping the sort after a full pass in which there are no exchanges.

```
public static void bubbleSort (String[] list)  
{  
    boolean sorted = false;
```

```

for (int top = list.length-1; top > 0 && !sorted; top--)
{
    sorted = true;
    for (int i = 0; i < top; i++)
        if (list[i].compareTo(list[i+1]) > 0)
        {
            sorted = false;
            String temp = list[i];
            list[i] = list[i+1];
            list[i+1] = temp;
        }
}

```

Although bubble sort uses an interesting technique and has a cute name, we do not recommend it. It is almost always slower than either of the sorts that we have already examined because it usually involves far more data movement than they require.

Exercises 10.5

1. Make tables like those shown in Example 2 to show the comparisons and exchanges that would take place in using a bubble sort to put the following data in ascending order.

3 8 3 2 7 5

2. What changes would have to be made to the `bubbleSort` method in order to make it sort values in descending order?
3. A modification of the bubble sort is the *cocktail shaker sort* in which, on odd-numbered passes, large values are carried to the top of the list while, on even-numbered passes, small values are carried to the bottom of the list. Make tables like those shown in Example 2 to show the first two passes of a cocktail-shaker sort on the following data.

2 9 4 6 1 7

4. Write a method `shakerSort` to implement the cocktail shaker sort algorithm to arrange an array of `double` values in ascending order. Use a boolean flag to stop processing once the items have been completely sorted.

10.6 Shellsort

Although insertion sort works well for short lists, it tends to slow down drastically with long lists. This is because of the large numbers of comparisons that are needed to find the correct positions for insertions of data. If, however, the elements of a list are almost in order, insertion sort is very fast, even for long lists because in this case only a few comparisons are needed to find the correct insertion point for each value. Shellsort, invented by Donald Shell in 1959, adapts insertion sort to produce a sorting technique that works well even with long lists that are randomly ordered. With Shell's adaptation of insertion sort, data can be moved long distances with only a few comparisons.

The sort uses the following idea. We say that a list is *k-sorted* if, starting at any point in the list, every k^{th} element is in order. A *k-sorted* list consists of *k* interleaved, sorted sublists. To illustrate the idea of a *k-sorted* list, let us see how a list can be 3-sorted.

Example 1

Suppose that we are given an unsorted list of integers.

34 21 40 12 27 18 29 13 25 17 11 38

This can be decomposed into three sublists with the first list containing the elements at positions 0, 3, 6, ..., the second containing the elements at positions 1, 4, 7, ..., and the third containing the elements at positions 2, 5, 8,

34 12 29 17
 21 27 13 11
 40 18 25 38

If we now sort each sublist, we obtain

```

      12      17      29      34
    11      13      21      27
      18      25      38      40

```

to give us a 3-sorted list.

```

12  11  18  17  13  25  29  21  38  34  27  40

```

What does this achieve? If we examine the original list of values in Example 1 together with the 3-sorted list and the list as it would be if it were completely ordered, we can see that in the 3-sorted list, many of the elements have been moved long distances and most of them are quite close to their final destination.

Unsorted:	34	21	40	12	27	18	29	13	25	17	11	38
3-sorted:	12	11	18	17	13	25	29	21	38	34	27	40
Sorted:	11	12	13	17	18	21	25	27	29	34	38	40

Without doing very much work to obtain the 3-sorted list, we have gone a long way toward our final goal.

Shellsort repeatedly uses insertion sort to create sorted sublists. Initially a k -sorted list is created using a large value of k so that values can be moved long distances. On subsequent passes, other k -sorted lists are formed using successively smaller values of k . On the final pass, the value of k is set to one; a 1-sorted list is completely sorted.

Example 2

The tables show the results of using Shellsort on the values

```

64  31  10  40  22  49  82  20  29  56  40  18  19  27  26

```

in which successive passes of the sort creates lists that are 7-sorted, 3-sorted, and 1-sorted.

Unsorted:	64	31	10	40	22	49	82	20	29	56	40	18	19	27	26
	20						26								64
		29						31							
			10						56						

First Pass

```

      40          40
        18      22
          19    49
            27      82

```

7-Sorted: 20 29 10 40 18 19 27 26 31 56 40 22 49 82 64

Second Pass

```

      20      27      40      49      56
        18      26      29      40      82
          10      19      22      31      64

```

3-Sorted: 20 18 10 27 26 19 40 29 22 49 40 31 56 82 64

Third Pass

1-Sorted: 10 18 19 20 22 26 27 29 31 40 40 49 56 64 82

In using Shellsort, we noted that the first pass should use a large value of k so that values that had to be moved long distances could do so quickly. We also noted that the last pass must use $k = 1$ in order to guarantee a completely sorted list. It is natural to ask: what is the best sequence of values of k ? Surprisingly, the answer to that question is not known, despite the fact that a good deal of work has been done on the problem. One sequence, however, that has been found to give good results is $\dots, 40, 13, 4, 1$. Working from the *right* end of this sequence, each term is one more than three times as large as the term on its right.

Exercises 10.6

1. Given the following data, show how they would appear after they have been 5-sorted.

26 37 21 41 63 19 61 72 55 29 47 18 26 22

2. Starting with the same set of data given in Question 1, show how they would appear after they had been 4-sorted.
3. In the text, we suggested that a good sequence of values of k ended with 40, 13, 4, 1. Give the next three smallest terms of this sequence.

4. How would you answer the following argument against using Shellsort? “The last step of Shellsort, using $k = 1$, is simply a normal insertion sort. Since Shellsort performs many preliminary steps before this final one, it must be slower than a single insertion sort.”
5. Suppose that you were going to write a version of Shellsort using the sequence of k -sorts suggested in the text. For a list containing n elements, the first value of k that should be used is the largest value in the sequence that is less than n .
 - (a) Write a sequence of statements that will initialize k correctly for a given value of n .
 - (b) Write a statement that will, for any value of k in the sequence, produce the next smaller value of k .
6. (a) Write a method `shellSort` to sort an array of `int` values in ascending order. In performing the k -sorts, use the sequence of values of k suggested in the text. Be sure to use insertion sort at each stage of the sort.
 - (b) Test your method by writing a complete program that first generates an array of 500 random `int` values in the range $[0, 999]$, prints this array (ten values per line), sorts the array using your `shellSort` method, and then prints the resulting array (ten values per line).
7. Experiment with using Shellsort with sequences for k other than the one given in the text, testing your sequences on large arrays of integers (generated as indicated in the previous question) and noting the time required by the sort in each case. To measure the time taken by a sort, you can use the method `currentTimeMillis` in the `System` class. The method has the signature

```
public static long currentTimeMillis()
```

It returns, as a `long` value, the number of milliseconds since midnight GMT on January 1, 1970. To use the method, you might write

```
long startTime = System.currentTimeMillis();
```

just before starting the sort and

```
int elapsedTime =
    (int)(System.currentTimeMillis() - startTime);
```

just after completing the sort. In choosing a sequence, remember that the last value of k must be one but, beyond this, there are no restrictions on the sequences. Can you find a sequence that performs

better than the one given in the text? (It is possible to beat the given sequence; we chose it because it is fairly simple and reliable.)

10.7 Comparing Algorithms

In this chapter we have looked at two searching algorithms and four sorting algorithms. We now turn our attention to the problem of comparing these algorithms, to try to decide which is “best”.

Before we start comparing the efficiency of algorithms, we must agree on some way of measuring efficiency. We do *not* want a measure that is dependent on the language in which the algorithm is coded or on the machine on which the program is executed. Instead, we want a measure of performance that depends only on the “size” of the problem. For most problems, there is usually some clear measure of the size. For searching and sorting lists of values, the size of a problem is usually taken to be the number of elements in the list.

Once we have a measure of the size of a problem, we then want to determine how a solution to a problem grows relative to the growth in its size. To measure growth of a solution, we often compare size to either the space required by the solution or the time required by the solution. (Frequently it turns out that there is a tradeoff between these two quantities; an algorithm that is relatively fast will use more space than one that is slower.) In this text, we will not be concerned with space considerations; we will restrict ourselves to a (somewhat informal) study of growth in time relative to the growth in size of our problems.

To begin, consider the sequential search algorithm developed in the first section of this chapter. Most of the time spent by the search is devoted to comparisons between the item sought and elements of the array being searched. If there are n elements in the array, then the maximum number of comparisons that would be required is n . If, on one machine, a single comparison takes c_1 time units, then, since most of the time taken by execution of the algorithm is devoted to comparisons, the total time for the search will be roughly $c_1 \times n$ time units. If we let this time be T_1 time units, then we have

$$T_1 \doteq c_1 n$$

If we change computers or programming languages, we might find that a single comparison now takes c_2 time units. In this case, the total time, T_2 ,

will be given by

$$T_2 \doteq c_2 n$$

In either case, the amount of time required by the sequential search is approximately proportional to n . If we let the time be T , then we *could* write this approximate proportionality in the form

$$T \propto n$$

Instead of this notation, however, we will usually write the relationship in the form

$$T \text{ is } O(n)$$

Reasonably enough, the use of O is called *big oh* notation. We can read the statement as “ T is big oh of n ” or “ T is order n ”

We should confess that we are, in fact, not really telling the truth about big oh notation. More advanced courses, dealing with analysis of algorithms, give a more precise definition.¹ For our purposes, however, we can think of the following relationships for a function f as being equivalent.

$$\begin{aligned} T &\doteq cf(n) \\ T &\propto f(n) \\ T &\text{ is } O(f(n)) \end{aligned}$$

In making our analysis of the time required by a sequential search, we supposed that *all* the elements of a list would have to be examined. This is known as a *worst case* analysis. If we wanted to examine the average time required by a sequential search, we might make the assumption that the data are randomly ordered so that, on average, we would have to examine about half of the elements to terminate a successful search. If the worst case gave

$$T \doteq cn$$

then the average case (under our assumptions) would be

$$T \doteq \frac{1}{2}cn$$

This means that T is still approximately proportional to n . Using our big oh notation, we have, once again,

$$T \text{ is } O(n)$$

¹Here is such a definition: A function $g(n)$ is said to be $O(f(n))$ if there exist constants $c > 0$ and n_0 , a positive integer, such that $g(n) \leq cf(n)$ for all $n \geq n_0$.

Analysis of the performance of binary search requires a somewhat different approach. As with sequential search, we use the number of comparisons to obtain an approximation of the rate of growth of time, T , as the size, n , of a list increases. Again, as with sequential search, in the worst case we must continue to examine elements until the size of the list is reduced to zero. Unlike the case with a sequential search, however, each time we make a comparison with a binary search, we eliminate approximately half of the remaining items. We let the total number of comparisons required for a list of size n be c_n . Since one comparison eliminates approximately half the elements in the list, we have

$$c_n \doteq 1 + c_{n/2}$$

Similarly

$$c_{n/2} \doteq 1 + c_{n/4}$$

Substituting the second equation in the first gives

$$c_n \doteq 2 + c_{n/4} = 2 + c_{n/2^2}$$

After one more comparison, we have

$$c_n \doteq 3 + c_{n/8} = 3 + c_{n/2^3}$$

After k comparisons, we have

$$c_n \doteq k + c_{n/2^k}$$

To actually determine the value of c_n from this equation, we note that exactly one comparison is required for a list of size one. Thus, if $n/2^k = 1$, $c_{n/2^k} = 1$ and hence

$$c_n \doteq k + 1 \text{ where } n/2^k = 1$$

Now, to obtain c_n in terms of n , we solve the equation $n/2^k = 1$ for k .

$$\begin{aligned} \frac{n}{2^k} &= 1 \\ 2^k &= n \\ k &= \log_2 n \end{aligned}$$

Because logarithms with base 2 appear so often in analysis of algorithms, we often abbreviate the expression $\log_2 n$ as simply $\lg n$. Substituting this

value for k in our equation for c_n gives us

$$\begin{aligned} c_n &= \lg n + 1 \\ c_n &\doteq \lg n \end{aligned}$$

Since the time required by the search will be roughly proportional to the number of comparisons, we have, finally

$$\begin{aligned} T &\propto \lg n \\ T &\text{ is } O(\lg n) \end{aligned}$$

To analyze the performance of selection sort we note that, once again, the primary activity during execution of the sort is comparison of items. The first pass requires $n-1$ comparisons to find the largest item, the second pass requires $n-2$ comparisons, the third pass requires $n-3$ comparisons, and so on until the last pass requires only one comparison. The total number of comparisons is

$$(n-1) + (n-2) + (n-3) + \cdots + 1$$

and we know from mathematics that the sum of this arithmetic series is

$$\frac{1}{2}(n^2 - n)$$

As with our searches, most of the time used by selection sort is spent doing comparisons. If we let T units be the total time taken by selection sort, then

$$T \propto \frac{1}{2}(n^2 - n)$$

Since we are dealing here with proportionalities, we can eliminate the constant multiplier to obtain

$$T \propto (n^2 - n)$$

or

$$T \text{ is } O(n^2 - n)$$

We can simplify this expression even further. To do so, let us compare values of the expression $n^2 - n$ with n^2 .

n	$n^2 - n$	n^2	$(n^2 - n)/n^2$
1	0	1	0.0
10	90	100	0.9
100	9 900	10 000	0.99
1 000	999 000	1 000 000	0.999
10 000	99 990 000	100 000 000	0.9999
100 000	9 999 900 000	10 000 000 000	0.99999
1 000 000	999 999 000 000	1 000 000 000 000	0.999999

As you can see from the table, as n gets larger, the effect of subtracting n from n^2 becomes (in relative terms) almost insignificant. For large values of n , the expressions $n^2 - n$ and n^2 are relatively very close as indicated by the ratios in the last column. In such a situation, where one term of an expression dominates any others, our big oh notation allows us to simplify our description of an algorithm's behaviour by eliminating the non-dominant terms. For the selection sort we can, therefore, write

$$T \text{ is } O(n^2)$$

In addition to comparing items, sorting also involves moving them around if they are out of order. Using a selection sort on a list with n items, we may have to perform $n - 1$ swaps to get items into their correct positions. If each swap takes s time units, then the total time for all swaps is $s(n - 1)$. This operation is, therefore, $O(n)$. Since the comparisons are $O(n^2)$, the time for comparisons dominates the total time and so selection sort is $O(n^2)$.

Both insertion sort and bubble sort are also $O(n^2)$ algorithms but insertion sort usually requires more data movement than selection sort and bubble sort usually requires *far* more data movement, making it slower than either selection sort or insertion sort. Shellsort can provide dramatic improvement in running time over any of the other three sorts that we have studied. Although no exact analysis of the performance of Shellsort has yet been possible, it has been found that with a good sequence of jump sizes, the time taken by Shellsort is approximately $O(n^{1.25})$. There are a number of sorts that have even better performance than Shellsort. We will study one such sort, quicksort, when we examine a technique called recursion in Chapter 11.

Although our performance measures are not linked to specific computers or programming languages, they can help us to determine actual performance times. This is illustrated in the next example.

Example 1

A selection sort running on a particular computer requires 0.003 s to sort 200 items.² Estimate the time that the sort would require on the same machine to sort 8000 items.

Since selection sort is $O(n^2)$, the time taken by the algorithm is roughly proportional to n^2 so that

$$T \doteq k \times n^2$$

where k is a constant for a particular environment. For two separate runs on the same computer, we would then have

$$\begin{aligned} T_1 &\doteq kn_1^2 \\ T_2 &\doteq kn_2^2 \end{aligned}$$

Combining these equations gives

$$\frac{T_2}{T_1} \doteq \frac{kn_2^2}{kn_1^2}$$

or

$$\frac{T_2}{T_1} \doteq \frac{n_2^2}{n_1^2}$$

Substituting the values that we know, we get

$$\begin{aligned} \frac{T_2}{0.003} &\doteq \frac{8000^2}{200^2} \\ T_2 &\doteq 4.8 \end{aligned}$$

The sort would take about 5 s to sort 8000 items.

Exercises 10.7

1. If an algorithm is known to be $O(n^2)$ and it takes t s to process a problem of size n , about how long would it take (in the same computing environment) to process a similar problem of size

²The values used in this example and in the questions of the following exercises are *not* intended to represent actual performance results of today's computers.

- (a) $2n$?
 - (b) $3n$?
 - (c) $5n$?
 - (d) $10n$?
2. If an algorithm is known to be $O(n^3)$ and it takes t s to process a problem of size n , about how long would it take (in the same computing environment) to process a similar problem of size
 - (a) $2n$?
 - (b) $4n$?
3. Binary search is known to be $O(\lg n)$. Suppose that a binary search requires c comparisons (in the worst case) to search a list of size n . What is the maximum number of comparisons that the search would require for a list of size
 - (a) $2n$?
 - (b) $4n$?
 - (c) $8n$?
 - (d) $64n$?
4. Suppose that, in a certain environment, selection sort requires 0.06 s to sort a list of length 500.
 - (a) Estimate the time that it would require to sort a similar list of length 50 000.
 - (b) Estimate the number of items that could be sorted in one hour.
5. Suppose that, in a certain environment, Shellsort requires 0.004 s to sort a list containing 1000 items.
 - (a) Estimate the time that it would take to sort a similar list of length 100 000.
 - (b) Estimate the number of items that could be sorted in one second.
6. Although Shellsort is more efficient than straight insertion sort, insertion sort might be faster for short lists because the algorithm is simpler. Suppose that, on a particular machine, both insertion sort and Shellsort require 0.05 s to sort 200 items. Find the approximate amount of time each one would take to sort 100 000 items.
7. Draw graphs showing the number of comparisons that would be required (in the worst case) by each of the following algorithms.
 - (a) sequential search
 - (b) binary search
 - (c) selection sort

10.8 Comparing Objects

Throughout this chapter, the examples we have shown for our sorts have almost invariably used primitive types. Often, however, we want to sort objects. As examples, we might want to order student record objects by student numbers while we might want to order objects representing shapes by size.

To sort objects, we can start by writing a sort that is applicable to any objects that we might consider to be sortable. Let us designate such objects as being of type `Sortable`. Of course, if we are going to sort these objects, we must decide what it means for one object of type `Sortable` to “precede” another object of the same type. Suppose, for now, that we have done this by writing an instance method for `Sortable` objects with the following signature

```
public boolean precedes (Sortable other)
```

The method `precedes` returns `true` if and only if its implicit object precedes `other`. Once we have a method that defines criteria for ordering `Sortable` objects, we can then write a sort for such objects.

Example 1

A selection sort for `Sortable` objects could have the form

```
public static void selectSort (Sortable[] list)
{
    for (int top = list.length - 1; top > 0; top--)
    {
        int largeLoc = 0;
        for (int i = 1; i <= top; i++)
            if (list[largeLoc].precedes(list[i]))
                largeLoc = i;
        Sortable temp = list[top];
        list[top] = list[largeLoc];
        list[largeLoc] = temp;
    }
}
```

Our goal is to make this method applicable to a wide variety of objects, unrelated except for the fact that each of them has some criterion for ordering the objects. To do so we must do two things:

- Specify the requirements that a class must satisfy if its objects are to be considered sortable. There is only one such requirement: any `Sortable` class must have a `precedes` method.
- Show how each class that we want to be considered `Sortable` should implement the `precedes` method.

For the first part, we use a structure that we have not yet seen in Java — an *interface*. Interfaces are very similar to the classes that we have been using; they can contain both fields and methods. The definition of the `Sortable` interface might take the following form.

```
interface Sortable
{
    public boolean precedes (Sortable other)
        ;
}
```

Notice in the definition the word `interface` instead of `class`. Notice also that the method of the interface has no body. This is characteristic of all interfaces; their methods never have bodies. All that the definition does is, in a sense, lay out the framework for what would be required of any class that wants to be considered to be of type `Sortable`.

For the second part, for each class that we want to be designated as `Sortable`, we must provide a `precedes` method with the signature given in the interface. This method must define how to determine the ordering of objects *of that type*. The next example illustrates how we can do this.

Example 2

Suppose that we want to be able to sort objects of the type `Fraction` that we used when we introduced the idea of an object. Recall that the `Fraction` class had two fields, `num` and `den`, representing the numerator and denominator of a fraction. There is a natural way to decide if one `Fraction` object precedes another: we simply examine the values of the fractions that correspond to the `Fraction` objects. To implement this, we rewrite the `Fraction` class as follows.

```

class Fraction implements Sortable
{
    private int num;
    private int den;

    public boolean precedes (Sortable s)
    {
        Fraction f = (Fraction)s;
        double thisValue = (double)num/den;
        double otherValue = (double)f.num/f.den;
        if (thisValue < otherValue)
            return true;
        else
            return false;
    }

    // other methods of the class

}

```

Notice the phrase `implements Sortable` in the first line. This tells Java that there is a method in the `Fraction` class that provides a body for the method `precedes` in the `Sortable` interface.

Notice also that the type of the parameter in the `precedes` method is `Sortable`. The `Sortable` interface specifies that any `precedes` method must accept a parameter that is of type `Sortable`. Within the method, we cast the parameter to an appropriate type of object (`Fraction` in this case) with the statement

```
Fraction f = (Fraction)s;
```

If we were writing a `precedes` method for a `Widget` class, we would have to cast the `Sortable` parameter to an object of type `Widget`.

With this revised definition of the `Fraction` class, we can now use the `selectSort` method of Example 1 to sort an array of `Fraction` objects because `Fraction` objects can also be considered as `Sortable` objects. If our program contained other classes that implemented `Sortable`, then arrays of objects of these classes could also be sorted using exactly the same method.

This idea is such a good one that a form of it is actually implemented

in Java. In the `java.lang` package, there is an interface called `Comparable` that contains the header of a single method called `compareTo` with the signature

```
public int compareTo (Object o)
```

Any class implementing the `Comparable` interface must supply a `compareTo` method that accepts some kind of `Object` as a parameter and returns an `int` value as specified in the following table

Relationship Between Objects	Value Returned
implicit object precedes explicit object	some negative integer
implicit object equals explicit object	zero
implicit object follows explicit object	some positive integer

We have already encountered the `compareTo` method in our study of strings. The header of Java's `String` class contains the phrase `implements Comparable` and the class contains a `compareTo` method for `String` objects.

Example 3

An insertion sort that could be used for any objects that implement the `Comparable` interface could take the following form.

```
public static void insertSort (Comparable[] list)
{
    for (int top = 1; top < list.length; top++)
    {
        Comparable item = list[top];
        int i = top;
        while (i > 0 && item.compareTo(list[i-1]) < 0)
        {
            list[i] = list[i-1];
            i--;
        }
        list[i] = item;
    }
}
```

If we wanted to apply this sort to an array of `Fraction` objects, all that we would have to do is add a `compareTo` method to our `Fraction` class and insert the phrase `implements Comparable` in the header of the `Fraction` class.

Example 4

The following code shows what must be added to the `Fraction` class to make it possible to sort `Fraction` objects using the sort in the preceding example.

```
class Fraction implements Comparable
{
    private int num;
    private int den;

    public int compareTo (Object o)
    {
        Fraction other = (Fraction)o;
        double thisValue = (double)num/den;
        double otherValue = (double)other.num/other.den;
        if (thisValue < otherValue)
            return -1;
        else if (thisValue == otherValue)
            return 0;
        else
            return 1;
    }

    // other methods of the class
}
```

Since the parameter of the `compareTo` method in the `Comparable` interface is `Object`, any `compareTo` method that we write must have a parameter of that type. Within the method, we cast the parameter to a reference to a `Fraction` object.

Using the `Comparable` interface, we have a technique that allows us to use a single sorting method to sort any type of *object* that we create but it cannot sort lists containing *primitive types* such as `int` or `double`. To include these, Java also offers a number of *wrapper classes* that are used, as their name suggests, to enclose a primitive type in an object. The identifiers of the wrapper types are usually the same as the corresponding primitive types but written with an initial upper case letter. For example, the wrapper class for `double` values is `Double` while the wrapper class for `byte` is `Byte`. The only exceptions to this are the wrapper classes for `int` and `char` whose identifiers are `Integer` and `Character`. All the wrapper classes (except `Boolean`) contain appropriate `compareTo` methods.

Example 5

The following program first creates three arrays: an array of `Fraction` objects, an array of `Integer` objects used as wrappers for `int` values, and an array of `String` values. After printing the contents of the original lists, it then uses a single sort method to sort all three arrays of objects and then prints the sorted results. The program assumes that the `Fraction` class contains the phrase `implements Comparable` in its header and that it contains definitions of the following methods: a `compareTo` method as required by the `Comparable` interface, a constructor that accepts a numerator and denominator as arguments, and a `toString` method that returns a string of the form "a/b".

Notice the use (in the `println` statements) of the method `intValue`. This method is used to recover `int` values from objects in the `Integer` wrapper class.

```
public class SortDemo
{
    public static void main (String[] args)
    {
        Fraction[] f = new Fraction[3];
        f[0] = new Fraction(3,4);
        f[1] = new Fraction(1,2);
        f[2] = new Fraction(2,3);

        Integer[] n = new Integer[3];
        n[0] = new Integer(7);
        n[1] = new Integer(5);
```

```

        n[2] = new Integer(2);

        String [] s = {"Jonathan","Callum","Melissa"};

        System.out.println("Original Values\n");
        for (int i = 0; i < f.length; i++)
            System.out.println(f[i] + "    " + n[i].intValue()
                               + "    " + s[i]);

        selectSort(f);
        selectSort(n);
        selectSort(s);

        System.out.println("\nSorted Values\n");
        for (int i = 0; i < f.length; i++)
            System.out.println(f[i] + "    " + n[i].intValue()
                               + "    " + s[i]);
    }

    public static void selectSort (Comparable[] list)
    {
        for (int top = list.length - 1; top > 0; top--)
        {
            int largeLoc = 0;
            for (int i = 1; i <= top; i++)
                if (list[largeLoc].compareTo(list[i]) < 0)
                    largeLoc = i;

            Comparable temp = list[top];
            list[top] = list[largeLoc];
            list[largeLoc] = temp;
        }
    }
}

```

The output from this program would be

Original Values		
3/4	7	Jonathan
1/2	5	Callum
2/3	2	Melissa
Sorted Values		
1/2	2	Callum
2/3	5	Jonathan
3/4	7	Melissa

In Example 5, the `compareTo` method operates differently depending on the type of object associated with it. The elements of the `list` array can be of type `String`, `Fraction`, or `Integer`. At execution time, Java determines what type of object is involved in the call and picks the appropriate form of the `compareTo` method. This extremely useful feature is an example of *polymorphism*, a word meaning “occurring in many forms”. Java’s interfaces allow us to introduce polymorphism very easily. To invoke polymorphism, an object can be associated with a method as an implicit parameter, an explicit parameter, or both (as in the case of `compareTo`).

Exercises 10.8

1. Consider a class `Student` that has a field defined by


```
private int studentNumber;
```

 Write a definition for a `compareTo` method for the `Student` class that returns `-1` if the `studentNumber` of its implicit parameter precedes that of its explicit parameter, `0` if the `studentNumber` fields have equal value, and `+1` otherwise.
2. (a) Define a class `Point` with `private double` fields `x` and `y` that represent the coordinates of a point. Your definition of the class should include features that permit the use of the `selectSort` method of Example 1 to sort an array of `Point` objects. For `Point` objects `p1` and `p2`, consider `p1` to precede `p2` if and only if the first point is closer to the origin than the second.
 (b) Change the definition of the `Point` class so that it implements Java’s `Comparable` interface.

10.9 Review Exercises 10

1. For the list

Bat Cat Cow Dog Elk Fly Fox Gnu Hen Owl Pig Rat Yak

trace the execution of a binary search as it looks for the value `Man` in the list. In your trace, note the bounds of the sub-array and the value in the list that is being examined on each pass. If a sub-array has no exact middle, choose the item to the immediate left of the middle.

2. Sequential search as given in the text does not require that the list of items be sorted but, if the list *is* sorted, then sequential search can be made more efficient. Write a method with header

```
public static int seqSearchSorted (String[] list,
                                   String item)
```

that uses the fact that `list` is sorted in ascending order to make the search more efficient. The method should, as usual, return the index in `list` of the first occurrence of `item` or `-1` if `item` is not in the list.

3. Rewrite the binary search method of Section 10.2 so that, if there is more than one element in the list whose value is equal to `item`, the method returns the index of the *first* such element in the list.
4. Suppose that the following list of values is to be sorted into ascending order.

5 1 4 8 9 6 3

Show the list as it would appear after the first *and* second pass of each sort.

a) selection sort b) insertion sort c) bubble sort

5. Suppose that the following list is to be arranged in ascending order.

86 13 14 61 54 70 75 26 50 12 53 29 70 65

Show the list as it would appear after one pass of Shellsort with $k = 5$.

6. Suppose that insertion sort is to be used to sort a list of n integers. Show that, in the worst case, the number of data movements required is $\frac{1}{2}n^2 + \frac{3}{2}n - 2$.
7. We might think of the process of shuffling (as used with decks of playing cards) as the opposite of sorting. To investigate shuffling, suppose that you are given an array called `list` of n `int` values. Initially, `list[i]` has the value i .
 - (a) Suggest a reasonably efficient algorithm that could be used to shuffle the numbers in `list`.
 - (b) Implement your algorithm as a Java method with heading
`public static void shuffle (int[] list)`
8. Write a program that reads up to 1000 positive integers. The values are not necessarily all different and they are not in any particular order. The program should continue reading input until it has read 1000 values or it has read the sentinel value, zero. The program should then print all the distinct positive integers read (in ascending order) together with a count of the number of occurrences of each number. As an example, input of

5 7 2 5 4 9 4 7 5 1 0

should produce output like the following.

Value	Frequency
1	1
2	1
4	2
5	3
7	2
9	1

Projects

9. Write a program to play the following game. Begin by displaying the ten digits 0 - 9 in some random order. Then repeatedly ask the person playing the game to choose a number n , after which the computer will reverse the first n digits in the sequence. The object of the game is for the player to put all the digits in ascending order

using as small a number of reversals as possible. Set up your program so that after each game it asks if the game should be repeated with the same starting sequence, played with a new starting sequence, or terminated.

10. In the card game called bridge, each of the four players is dealt thirteen cards. To help players to evaluate the worth of their hands, it is common to assign a numerical value to each hand. One widely used system assigns values as follows:

- 4 points for each ace
- 3 points for each king
- 2 points for each queen
- 1 point for each jack
- 3 points for a void (no cards in a suit)
- 2 points for a singleton (one card in a suit)
- 1 point for a doubleton (two cards in a suit)

Write a program that will read a string representing a bridge hand. The string should consist of 26 characters, two characters for each card. For each card, the first character will represent the denomination (using A for ace, K for king, Q for queen, J for jack, T for ten, and 2 to 9 for the other denominations) and the second character will represent the suit (S for spades, H for hearts, D for diamonds, and C for clubs). For example, the string

4DTCKC5H7CQSAH4H6SJHAC9H2H

would represent a hand containing the four of diamonds, ten of clubs, king of clubs, and so on.

Your program should repeatedly ask the user if another hand should be evaluated. If so, the program should prompt the user to submit a hand, read the input, and determine whether or not the input is valid. If the input is not valid, the program should reject it and print a message. If it is valid, the program should evaluate the hand and print a display of the hand together with its value. The display should be arranged according to the following rules. Spades should be listed on the first line, hearts on the second line, diamonds on the third line, and clubs on the fourth line. Within each line, the cards of a suit should be arranged in descending order of value (ace, king, queen, jack, 10, 9, ..., 2) with each value separated by one

blank. For example, with the input shown above, the output would be

Spades	queen 6
Hearts	ace jack 9 5 4 2
Diamonds	4
Clubs	ace king 10 7
Value:	17 points

11. The Tenants' Rights Bureau wants you to assist them in monitoring complaints against apartment landlords. At the end of each month, for each of the buildings in its files, the Bureau will give you the building name, the number of units in the building, the number of complaints received this month, and the name of the landlord. You may assume that there are no more than one hundred buildings in the Bureau's files. The Bureau wants you to produce the following lists:
 - (a) an unsorted list of the raw data, for checking purposes,
 - (b) a listing of the data, with suitable headings, sorted alphabetically by building name,
 - (c) a listing of the data (again with suitable headings) sorted alphabetically by landlord,
 - (d) another table, in alphabetical order by landlord, in which there is only one entry for each landlord, each entry showing the total number of units owned by that landlord and the total number of complaints against that landlord,
 - (e) the name of the worst landlord of the month, decided by finding the landlord having the highest percentage of complaints for total units owned.